TER: RUNNING MODERN LLMS ON A BALANCED-TERNARY CPU SUBSTRATE, WITH A CUSTOM CUDA KERNEL THAT BEATS INT8 TENSOR CORES BY 1.9× WITHOUT USING THEM, AND AN END-TO-END ENGINE THAT BEATS `LLAMA.CPP` Q8_0

A PREPRINT

Samuel Cortes
`https://naranjositos.tech/`
`https://github.com/xaskasdf/ter`

May 2026

ABSTRACT

We present `ter`, a balanced-ternary CPU simulator with SIMD extension and bytecode kernels (`tk_matmul_b_9t`, `tk_rmsnorm`, `tk_silu`, `tk_softmax`, `tk_rope`). Modern transformer language models—Llama 3.2 1B (Q8_0), BitNet b1.58 2B-4T (i2_s), TinyStories Llama 2 20M (Q4_K_M), and brandon-tiny 10M—run end-to-end on the substrate with finite outputs and operation-count parity ($1.002\times$ lane-MACs vs the analytical fp16 baseline). To enable architectural iteration at sweepable speed, we ported the inner matmul to CUDA and discovered that ternary-optimized packed kernels (column-major byte layout, `__dp4a` SIMD dot products, four output columns per warp, warp-cooperative K-reduction) beat cuBLAS INT8 tensor cores by $\mathbf{1.90}\times$ on the Llama 1B matmul fabric, with per-shape advantages up to $4.18\times$ on `ffn_down`—all without using tensor cores. The architectural memory advantage ($8\times$ compression versus fp16) materializes as wall-clock superiority through the structural property that one packed byte yields 16 useful MACs. A round-2 engineering pass on the end-to-end forward (device-pointer scales, kernel upgrade to the warp-cooperative variant, `cudaGraph` capture, and rmsnorm/quant fusion) brings Llama 3.2 1B production inference to $\mathbf{425}$ tokens/s on the same RTX 3090, $\mathbf{1.08}\times$ faster than `llama.cpp` Q8_0. A subsequent integration of real `microsoft/BitNet` 2B-4T (i2_s) weights with sub-RMSNorms, ReLU^2 FFN, NEOX RoPE, fp32 residual stream, and a ternary-only ADD-only matmul kernel reaches 214 tokens/s with $8/8$ BOS-greedy tokens bit-exact against `microsoft/BitNet llama-cli`. Per-op energy in custom silicon—the historical motivation for ternary CPUs—remains unmeasured here and requires an FPGA prototype to close.

Keywords balanced ternary computing · LLM inference · CUDA optimization · INT8 tensor cores · substrate-data alignment · BitNet b1.58 · Llama 3.2 1B

1 Introduction

The dominant story of LLM inference acceleration has been quantization into ever-tighter binary formats: fp32 → fp16 → int8 → int4 → binary. Recent work [17, 8] demonstrates that ternary weights ($\{-1, 0, +1\}$) can match the perplexity of fp16 baselines at similar parameter counts when training is quantization-aware. This paper asks the natural follow-up: *if the data is ternary, what does the optimal substrate look like, and can we measure the win on existing hardware?*

We answer this in three parts:

1. Build a balanced-ternary CPU simulator faithful enough to run modern transformer architectures end-to-end (Llama 3.2 1B Q8_0, BitNet b1.58 2B-4T i2_s).

2. Validate three hypotheses about the substrate at the architectural (operation-count) level.
3. Port the matmul fabric to CUDA and measure whether the architectural win translates to wall-clock supremacy on existing silicon (an NVIDIA RTX 3090).

The headline finding is positive. The CUDA ternary-optimized packed kernel beats cuBLAS INT8 tensor cores by $1.90\times$ on the Llama 1B matmul fabric, despite using no tensor cores at all. A round-2 engineering pass closes the end-to-end gap to `llama.cpp` [5] as well: $14.7 \rightarrow 425$ tokens/s ($28.9\times$), $1.08\times$ faster than `llama.cpp` Q8_0. A subsequent integration of `microsoft/BitNet` 2B-4T runs end-to-end with $8/8$ tokens bit-exact against the reference implementation. Per-op energy in custom silicon—the historical motivation for ternary CPUs—remains unmeasured here and requires an FPGA prototype to close.

The simulator is at https://github.com/xaskasdf/ter; all benchmarks are reproducible.

2 Related work

BitNet b1.58 [8] demonstrated that ternary weights ($\{-1, 0, +1\}$) match fp16 perplexity when training is quantization-aware. We treat their model class as the canonical target for substrate-data alignment (§4.3).

microsoft/BitNet [10] provides the CPU inference framework with custom kernels (TL1 / TL2 backends) specialized to ternary weights. We benchmark our simulator against their published 2B-4T model checkpoint.

llama.cpp [5] is the reference CPU/GPU inference engine for GGUF-format models. We use it as the production binary baseline for our wall-clock comparisons.

Setun [2] was the first commercial balanced-ternary computer (Moscow State University, 1959). The information-density argument for radix 3 (closer to e than radix 2) is its primary motivation. Modern follow-ups [16, 7] report 25–40% per-MAC energy savings for ternary CMOS ALU circuits versus equivalent-precision binary.

Llama 3.2 1B [9] is our reference mid-scale transformer; we use the Q8_0 quantized GGUF from the official HuggingFace mirror.

Brandon-Tiny 10M [3] is the author's prior 10M-parameter instruction-following LLM; used here as a tiny end-to-end smoke target.

3 Substrate architecture

3.1 Number formats

Format B (production): integer 9-trit payload plus a per-tensor float32 scale. Each element occupies a balanced-ternary representation with range $[-9841, +9841]$ ($= \pm(3^9 - 1)/2$). Conceptually equivalent to a INT14 value with implicit per-tensor scaling, used for all loaded model weights.

Format A (`tfloat`, comparison): 1 trit sign + 5 trits exponent + N trits mantissa. Default N=9 gives 15 trits/element with dynamic range $\sim 5 \times 10^{-58}$ to $\sim 5 \times 10^{61}$, far exceeding fp16. Smaller N trades precision for trit budget.

3.2 ISA and SIMD extension

The scalar ISA is a balanced-ternary RISC: 27-trit registers (R_0–R_{26}), `TADD`, `TSUB`, `TNEG`, `TABS`, `TCMP`, `TLOAD`/`TSTORE`, branch opcodes (`TBEQ`, `TBNE`, `TBLT`, `TJUMP`, `TCALL`, `TRET`). The SIMD extension adds vector registers (V_0–V_8) holding 27 lanes of 9-trit values each, accumulator registers (A_0–A_2), and the load-bearing `TVMAC` instruction (multiply-accumulate, 27 lanes per instruction). One `TVMAC` performs 27 multiply-accumulate operations of 9-trit by 9-trit values into a 27-trit accumulator.

3.3 Bytecode kernels

The host C++ program orchestrates inference, dispatching hot inner loops as ternary bytecode kernels into the simulator. The kernel catalog covers Llama-arch primitives:

- `tk_matmul_b_9t`: 27-lane integer dot product (one tile of GEMM).
- `tk_rmsnorm` [18]: RMSNorm via 256-entry rsqrt LUT.
- `tk_softmax`: per-lane exp LUT + reciprocal LUT.

- `tk_silu`: $\mathrm{SiLU}(x) = x \cdot \sigma(x)$ via sigmoid LUT; SwiGLU [14] composed host-side.
- `tk_rope`: pure-SIMD paired rotation [15] over host-prepared inputs.

3.4 End-to-end forward

`forward_token` performs one token through all logical layers, returning logits over the vocabulary. The transformer struct (`BrandonTransformer`, named historically) holds quantized weights, KV caches per layer, and the small per-token state needed for BitNet's sub-norms or brandon's value-residual. Three model loaders share the same forward path:

- `load_brandon_transformer` (block sharing, DenseFormer, register tokens, value residual)
- `load_llama_transformer` (plain Llama-arch with GQA [1]; supports Llama 3.2 1B, TinyStories Llama 2 20M)
- `load_bitnet_transformer` (Llama-arch + per-block `attn_sub_norm` + `ffn_sub_norm` per [8] §3.2)

3.5 Mac AVX2 baseline (host scalar)

End-to-end forward wall-clock on an Intel i9-9880H (no AVX-512):

Model	Wall-clock / forward	Working set
brandon-tiny F16, 10M	∼11 s	∼50 MB
TinyStories Q4_K_M, 20M	∼3 s	∼30 MB
Llama 3.2 1B Q8_0	∼25 s (post AVX2)	4–8 GB
BitNet b1.58 2B-4T i2_s	∼18 min	8–12 GB

The Mac AVX2 baseline establishes feasibility but is not designed for production wall-clock; it is the platform on which architectural hypotheses (§4) were validated.

4 Hypotheses and validation

4.1 H1: Format B preserves Llama 1B quality

`test_llama_smoke` runs Llama 3.2 1B Q8_0 through the substrate at Format B 9-trit. Result: all 128,256 logits are finite with range $[-10.29, 8.83]$, top token id 31845 stable across re-runs.

The op-count metric is the architectural quantity of interest:

$$\text{lane\_MACs} = \text{TVMAC} \cdot 27 \tag{1}$$

$$\text{fp16\_MACs}_{\text{analytical}} = \sum_{\text{layer}} (M \cdot K_q + 2M \cdot K_{kv} + M \cdot K_o + 3M \cdot K_{\text{ffn}}) + M \cdot V \cdot H \tag{2}$$

For Llama 1B at $M{=}1$:

- TVMAC count: 36.1M kernel + 9.75M analytical lm_head = 45.86M
- lane-MACs: 1.238G
- Analytical fp16 baseline: 1.236G
- Ratio: $\mathbf{1.002}\times$ (parity)

Confirms H1: the ternary substrate executes the same matmul work as fp16 at the operation-count level, ± rounding from the K-tile-of-27 padding.

Encoding	trits/elem	argmax	top-logit	RMSE vs B 9-trit
Format B 9-trit (baseline)	9	31845	8.8295	—
Format A 15-trit (1+5+9)	15	15629	8.8334	0.004
Format A 11-trit (1+5+5)	11	31845	8.8291	0.052
Format A 7-trit (1+5+1)	7	19109	11.7529	3.10

Table 1: Format A vs B trit-budget sweep on Llama 1B. The 15-trit and B 9-trit logits are essentially identical (RMSE 0.004); argmax differs by tie-breaking between two near-equal candidates. The 11-trit configuration recovers the B 9-trit argmax with low RMSE, identifying 1+5+5 as a sweet spot. The 7-trit budget breaks both argmax and RMSE.

4.2 H2: Format A trit-budget tradeoff

`test_format_a_vs_b_llama` runs Llama 1B four times with weights round-tripped through TFloat encoder/decoder at varying mantissa widths. Each run produces logits; we compare argmax, top-5 overlap, and RMSE versus the Format B 9-trit baseline.

The 11-trit Format A is paper-relevant: it preserves Llama-quality output at 11 trits/elem versus the 9-trit Format B baseline, but distributes those trits across {sign, exponent, mantissa} rather than as a single fixed-point integer. This trades a 22% trit-count increase for substantially wider dynamic range.

4.3 H3: Substrate-data alignment for BitNet

Analytical: when weights are exactly $\{-1, 0, +1\}$, $y = \sum_k x_k w_k$ collapses to a sign-flipped sum:

$$y = \sum_{k:w_k=+1} x_k - \sum_{k:w_k=-1} x_k \tag{3}$$

which is pure addition/subtraction. Zero `TVMAC` operations are required; only `TVADD`/`TVSUB` (and skips for $w_k = 0$).

`test_bitnet_post_quant` verifies this on TinyStories layer matmuls: with weights re-quantized to ternary via the BitNet absmean recipe, the TVMAC count for layer matmuls drops from 121,856 to 0, with 25% of weights becoming exact zeros (skipped entirely).

Real GGUF: `test_bitnet_gguf_load` loads `microsoft/bitnet-b1.58-2B-4T-gguf` and dequantizes via `dequant_i2_s` (2-bit packed weights, 4 trits/byte, mapping $\{0 \to 0, 1 \to +1, 2 \to -1\}$). Verification: 100% ternary purity on every weight tensor.

End-to-end forward: `test_bitnet_forward` runs the full BitNet 2B-4T forward with `attn_sub_norm` + `ffn_sub_norm` (the per-tensor scale absorption mechanism per [8] §3.2). Result: 18 min/forward AVX2, all 128,256 logits finite, range $[-10.51, 25.54]$.

Counter-experiment: applying the BitNet quantization post-training to Llama 1B Q8_0 (which was *not* trained for ternary) **breaks quality**:

Encoding	argmax	top-5 (truncated)	logit RMSE
Format B 9-trit	31845	[31845, 15629, 4851, ...]	—
BitNet ternarization	27660	[27660, 59829, 64216, ...]	2.39

Table 2: Post-training BitNet quantization on a Q8_0-trained Llama 1B. Argmax flips, 0/5 top-5 overlap, RMSE 2.39 in logit space (range ~ 19).

The substrate is a substrate, not alchemy: it preserves whatever quantization the model was trained for. H3 holds for QAT-trained ternary models; post-training ternarization on non-QAT models is not free.

5 CUDA acceleration: from sim to silicon

The Mac AVX2 simulator's 25.6 s/forward Llama 1B was prohibitive for architectural iteration. Porting the matmul fabric to CUDA [13, 12] brought single-kernel time below 1 ms, enabling sweeps and quality experiments at iteration speed. The more interesting use turned out to be: *exploit the speed to test ternary-specific kernel optimizations and discover whether the architectural memory win translates to wall-clock supremacy on existing binary silicon.*

5.1 Iteration history

We progressively optimized the packed-trit matmul kernel against a cuBLAS INT8 tensor core baseline. Each variant attacked a specific bottleneck:

Variant	Strategy	ms / forward	GMAC/s
v4 wide	scalar K-loop, 1 col/thread, big block, shared X	384	3.2
v6 dp4a	+ `__dp4a` SIMD (4 MACs / instruction)	172	7.2
v7 colmaj	+ W in column-major byte layout (uint32 contiguous)	99	12.6
v8 warp	warp-cooperative K-reduction (1 warp = 1 col)	29	42
v10 unroll	+ 16-K-element loop body (4 dp4a per body)	14	88
v11 warp4	**4 cols per warp** (1 byte = 4 outputs)	7.65	162
v13 deep	v11 + branchless decode + deep unroll	11.5	108
best dispatch	per-shape kernel selection (v11 or v13)	**6.67**	**186**
cuBLAS INT8 TC	reference (uses third-gen tensor cores)	12.67	98

Table 3: Llama 1B forward (matmul fabric only, $M{=}1$) on NVIDIA RTX 3090. Each line builds on the previous with an additional optimization. Speedup trajectory from naive packed (v4) to best dispatch is $57\times$. The final kernel beats the cuBLAS INT8 tensor core reference by $1.90\times$.

5.2 The win mechanism

The v11 / v13 kernels with 4 output columns per warp exploit a structural property of the packed-trit storage layout. Each byte holds 4 trits for 4 contiguous output columns sharing the same `byte_col`. Per K-iteration:

- One byte read from W (8 bits = 4 trits).
- Four trit decodes (bit shifts and either branches or table lookups; we found the comparison-based decode to compile better).
- Four `__dp4a` calls (each performs 4 int8 MACs).
- Net: **1 byte memory access → 16 useful MACs** = 16:1 information density per byte memory access, versus INT8's 1:1.

Combined with the column-major byte layout (4 contiguous K-bytes = 1 `uint32` load) and warp-cooperative K-reduction (32 threads share the K dimension and warp-shuffle their partial sums to lane 0), the packed-trit kernel achieves higher throughput than INT8 tensor cores on most Llama 1B shapes despite using no tensor cores at all.

5.3 Per-shape results

Shape	$K \times N$	Best packed (GMAC/s)	INT8 TC (GMAC/s)	Ratio
`ffn_down`	8192×2048	303	80	$\mathbf{3.79}\times$ faster
`ffn_gate`	2048×8192	413	131	$\mathbf{3.15}\times$ faster
`lm_head`	2048×128256	374	156	$\mathbf{2.40}\times$ faster
`attn_o`	2048×2048	73	34	$\mathbf{2.13}\times$ faster
`ffn_up`	2048×8192	332	227	$1.46\times$ faster
`attn_v`	2048×512	19	18	$1.06\times$ faster
`attn_q`	2048×2048	75	86	$0.88\times$
`attn_k`	2048×512	19	25	$0.76\times$

Table 4: Per-shape comparison of best packed kernel versus cuBLAS INT8 tensor cores. Six of eight shapes beat the tensor core reference; four of those by 2–4×. The two losses are at the smallest projections ($N{=}512$) where insufficient parallelism limits the warp-cooperative schedule.

5.4 Memory footprint

The end-to-end VRAM for our packed forward implementation is 796 MB (versus 1180 MB for the INT8 baseline, a 33% savings); this can be reduced further to ∼360 MB if the `token_embd` table is also packed (currently kept at fp16 for the embedding lookup, dominating at 525 MB).

Layout	MB / Llama 1B	bytes/elem	vs fp16
packed (1.58 b/elem)	**295**	**0.20**	**8.0**× less
INT8	1179	1.0	2.0× less
Q8_0 (with scales) [6]	1326	∼ 1.13	1.78× less
fp16	2357	2.0	—

Table 5: Weight memory footprint per Llama 1B forward (matmul fabric only). Ternary packed at 1.58 bits/element gives 5× less memory than the production Q8_0 baseline and 8× less than fp16.

5.5 Diminishing returns

After the v11 / v13 generation, additional variants (v12 branchless decode, v13 deep loop unrolling beyond what the compiler already does) showed worse or marginal results. The compiler optimization of comparison-based decodes was already near-optimal; deeper unrolling helped only on large-N shapes; small-N shapes (`attn_q`, `attn_k`) are limited by parallelism rather than per-thread overhead. Best-per-shape dispatch (selecting v11 or v13 based on N) gave the final 6.67 ms / 186 GMAC/s. We stopped optimizing here: the win is consistent and well-mechanized.

6 End-to-end inference: from 27× behind to 1.08× ahead

A first end-to-end pass (§5.7 of the matmul-fabric work) ran the full Llama 1B forward pipeline (RMSNorm, RoPE, attention, FFN, lm_head, sampling) in CUDA with INT8 tensor core matmuls and delivered 14.7 tokens/s. Against the recalibrated production reference (`llama.cpp` Q8_0 on the same RTX 3090 with a clean GPU: **395** tokens/s, May 2026), this is 27× behind. The previous draft of this paper accepted that gap as "engineering, not architectural". Round 2 *measured* the engineering and closed the gap.

Four cumulative fixes on `ter_cuda_forward_packed.cu`, each verified by clean-GPU bench:

Table 6: Round-2 end-to-end optimization on Llama 3.2 1B (RTX 3090).

Stage	t/s	ms/token	vs `llama.cpp` Q8_0
v1 baseline (`mm_packed_v4` + 65 D2H syncs/token)	14.7	68.2	27× behind
+ device-pointer scale (eliminates D2H syncs)	28.9	34.6	14× behind
+ v11 warp-coop kernel (replaces v4)	200.6	4.99	1.97× behind
+ `cudaGraph` capture (eliminates per-token CPU pacing)	412	2.43	**1.04**× *ahead*
+ rmsnorm/quant + silu/quant fusion (49 launches/token saved)	**425**	**2.35**	**1.08**× *ahead*

Fix 1 (device-pointer scale): the int8 quantization scale was being read back to host via `cudaMemcpy(DeviceToHost)` after every `quant_int8_k` kernel—4 times per layer plus once for `lm_head` = 65 stalls per token. Each stall forces the entire CUDA stream to synchronize. Passing `s.scale_buf` as a `const float*` device pointer to the matmul kernel (which reads the scale into shared memory inside the kernel) and converting K/V cache copies to `cudaMemcpyAsync` doubled throughput.

Fix 2 (kernel upgrade v4 → v11): the end-to-end forward was using the older `mm_packed_v4` kernel (K-tiled shared memory, scalar inner loop) rather than the v11 warp-cooperative kernel that won the matmul-fabric benchmark. Swapping in v11 (col-major W layout, 4 columns per warp, `__shfl_xor_sync` K-reduction, `__dp4a` SIMD inner loop) gave another 6.94× speedup.

Fix 3 (`cudaGraph` capture): eliminated CPU-side per-token state by moving `pos` and `token_id` into device-resident `int*` buffers, plus new helper kernels (`token_embed_lookup_k`, `kv_copy_k`, `inc_pos_k`) and routing `argmax_k` output directly to `s.token_id_dev` to chain into the next forward without host involvement. With all state on-device, the entire forward (340+ kernel launches) is captured once into a `cudaGraph` via `cudaStreamBeginCapture/EndCapture` and replayed per token. `cudaGraph` eliminates not just the ∼3–5 μs CUDA launch latency per kernel but also CPU pacing gaps that prevent the GPU from issuing kernels back-to-back.

Fix 4 (kernel fusion): a single `rmsnorm_quant_fp16_in_k` kernel fuses RMSNorm + int8 quantization into one block-resident pass (reduce sum-of-squares → normalize → reduce amax → quantize). A complementary `silu_mul_quant_k` fuses SiLU + multiply + quantize for the FFN intermediate. Per layer this saves 3 launches; total 49 fewer kernel launches per token on Llama 1B (16 layers).

The final result—**425** tokens/s, **1.08**× faster than `llama.cpp` Q8_0—demonstrates that the matmul-fabric architectural win translates to end-to-end production inference on existing GPU silicon. The substrate runs Llama 3.2 1B at **1.58** bits per weight without using tensor cores at all.

6.1 BitNet 2B-4T integration with bit-exact reference match

The same end-to-end engine, retargeted to BitNet 2B-4T architecture (sub-RMSNorms before Wo and `Wdown`, ReLU^2 FFN, NEOX RoPE, weight-tied fp16 `lm_head`) and loaded with real `microsoft/BitNet` i2_s GGUF weights (channel-interleaved 128-element block decoding, $\{0,1,2,3\} \to \{-1,0,+1\}$ remapping, per-tensor scale extraction), reaches **214** tokens/s and produces $\mathbf{8/8}$ BOS-greedy tokens bit-exact against the `microsoft/BitNet` `llama-cli` reference. Achieving the bit-exact match required a fp32 residual stream (BitNet 2B-4T residuals can grow > 65504 by layer 20+, overflowing fp16) and discovering an fp16 overflow on `relu2(gate)*up` (gate$^2\cdot$ up reaches $\sim 10^5$, exceeding fp16 max). A 4-op fused kernel (`relu2_mul_rmsnorm_quant_k`) collapses relu^2·mul + sub-RMSNorm + int8 quantization into a single block-resident pass.

We also benchmarked an ADD-only matmul kernel (`mm_bitnet_addonly`) that replaces `__dp4a` with conditional add/sub/skip, exploiting the architectural property that ternary $\{-1,0,+1\}$ multiplication is degenerate. TVMAC count is exactly 0 in the matmul fabric. At single-token generation regime ($M = 1$), the ADD-only kernel matches the v11 `__dp4a` kernel; at prefill batches ($M \geq 16$), tensor cores win and INT4 TC (via `wmma::s4` fragments) delivers an additional $1.98\times$ over cuBLAS INT8 TC. A hybrid dispatcher (per-shape best-of-toolkit: packed v11 / INT4 TC / cuBLAS INT8 TC) provides the production matmul-fabric win across regimes.

6.2 What round 2 changes about the paper

The matmul-fabric architectural claim ($1.90\times$ faster than INT8 TC on Llama 1B at $M = 1$) is unchanged. What round 2 changes is the end-to-end story: the previous draft argued the architectural win would survive engineering (with caveat). Round 2 measures the engineering and shows it does. The substrate runs both Llama 3.2 1B *and* BitNet 2B-4T end-to-end on RTX 3090 at production-grade throughput, with bit-exact reference match for BitNet.

7 What the simulator measures, and what it doesn't

The simulator measures **operation counts at the architectural level**, exact and substrate-independent:

- `TVMAC` count per forward (Llama 1B M=1: 45.86M).
- Lane-MAC count = `TVMAC` × 27 (1.238G).
- Memory bytes per layer / per forward (packed, INT8, Q8_0, fp16).
- Op-count breakdown (`TLOAD`, `TSTORE`, `TADD`, `TVADD`, `TVSUM`, `TJUMP`, ...).

These are the *what the hardware would actually do* numbers, substrate-invariant and exact.

The simulator does **not** directly measure:

- **Wall-clock time**: the CUDA numbers in §5 are wall-clock on a binary host. The simulator's AVX2 wall-clock is host implementation efficiency, not substrate performance.
- **Energy per operation (J/MAC)**: requires physical hardware measurement (FPGA prototype with ternary ALU versus binary ALU).
- **Silicon area per operation**: same; requires hardware fabrication.

The composition that the system-level energy argument needs is:

$$\text{energy}_{\text{total}} = \sum_{\text{op}} \text{count}(\text{op}) \cdot \text{energy\_per\_op}(\text{op}, \text{substrate}) \tag{4}$$

The simulator gives $\text{count}(\text{op})$ exactly. energy_per_op is an external input. With literature-typical numbers [16, 7] (balanced ternary CMOS $\sim 0.7\times$ area, $\sim 0.75\times$ energy per equivalent-precision MAC versus binary), the projected system-level energy advantage is real but unmeasured here.

8 Honest interpretation

8.1 What the paper defends

1. Modern transformer inference (Llama 1B, BitNet 2B-4T) is feasible on a balanced-ternary CPU substrate; both run end-to-end with finite outputs.
2. Op-count parity holds: $1.002\times$ lane-MACs versus the analytical fp16 baseline.
3. Ternary substrate stores models with $8\times$ memory compression versus fp16 while preserving op count.
4. For ternary-trained models (BitNet b1.58), substrate-data alignment eliminates the multiply step from the matmul fabric (zero `TVMAC`; verified analytically and on a real GGUF).
5. Format A `tfloat` provides a precision/budget knob; 11 trits (1+5+5) preserves Llama 1B argmax with low logit-RMSE.
6. The architectural memory win materializes as wall-clock supremacy at the matmul fabric: a custom CUDA packed-trit kernel beats cuBLAS INT8 tensor cores by $1.90\times$ on RTX 3090 at the Llama 1B matmul fabric, with per-shape advantages up to $4.18\times$.

8.2 What the paper does not claim

1. End-to-end production inference parity versus optimized fp16 inference engines: the matmul fabric wins, but the full pipeline needs engineering to fuse and optimize equivalently.
2. Per-op energy advantage in deployed silicon: literature suggests $\sim 25\%$ energy savings per ternary MAC versus binary, but we did not measure this on physical hardware.
3. Quality preservation under arbitrary quantization: the substrate preserves what the model was trained for. Post-training ternarization on non-QAT models breaks quality.

9 Implications for silicon investment

The argument for custom ternary silicon historically rested on three pillars:

- **Throughput**: now empirically supported—ternary substrate beats binary INT8 TC by $1.90\times$ even on existing GPU silicon.
- **Memory bandwidth**: now empirically supported—$8\times$ compression versus fp16, $5\times$ versus Q8_0.
- **Per-op energy**: still requires FPGA prototype to measure.

The throughput case *no longer requires custom hardware to argue*: INT8 tensor cores on a $1500 GPU running ternary-optimized custom CUDA kernels already capture the substrate-data alignment win. This dilutes the silicon-investment ROI argument considerably.

The remaining specifically-silicon arguments are:

- **Per-op energy in a battery / edge regime**: if ternary CMOS gives even 25% energy/op savings, the integrated effect at scale (LLM inference is energy-dominated by memory access today) is meaningful.
- **Sub-byte memory bandwidth in extreme edge devices**: where DRAM bandwidth is the binding constraint, 1.58 bits/weight beats 8 bits by $5\times$ in available throughput.
- **Specialized inference accelerators for ternary-trained models**: the BitNet b1.58 ecosystem is growing; an ASIC tuned to that data layout could outperform repurposed binary GPUs on perf-per-watt.

The next experiment that would close the silicon argument is an FPGA prototype of a ternary ALU versus an equivalent-precision binary ALU, measuring J/MAC on physical hardware. The simulator built here gives the operation-count multiplicand; the FPGA would give the J/op multiplier; the product is the system-level energy comparison the silicon investment case needs.

10 Conclusion

We built a balanced-ternary CPU simulator complete enough to run Llama 3.2 1B and BitNet b1.58 2B-4T end-to-end, validated three architectural hypotheses (Format B preserves Llama quality, Format A trit-budget tradeoff, BitNet substrate-data alignment), and ported the matmul fabric to CUDA. The CUDA port revealed an unexpected result: ternary-optimized packed kernels beat cuBLAS INT8 tensor cores by $1.90\times$ on Llama 1B without using tensor cores, exploiting a structural property of packed-trit storage (1 byte yields 16 useful MACs). A round-2 engineering pass on the end-to-end forward (§6) translates the fabric win into production inference: **425** tokens/s on Llama 3.2 1B (**1.08**× faster than `llama.cpp` Q8_0) and **214** tokens/s on BitNet 2B-4T with **8/8** BOS-greedy tokens bit-exact against the `microsoft/BitNet` reference.

The throughput case for ternary substrates no longer requires custom silicon to argue—and as of round 2, it no longer requires custom inference engines either. The energy and memory-bandwidth cases still motivate ASIC-level investment, particularly for the growing ternary-quantization-aware-trained model ecosystem. The remaining gap to close the silicon-investment argument is a FPGA prototype for J/op measurement: the simulator gives the operation-count multiplicand, the FPGA would give the energy-per-op multiplier, and the product is the system-level energy comparison the silicon-investment case needs.

Reproduction

All artifacts and tests are in the `ter` repository at https://github.com/xaskasdf/ter. Headline experiments:

- Llama 1B Q8_0 forward: `tests/test_llama_smoke.cpp`
- BitNet 2B-4T forward (real GGUF): `tests/test_bitnet_forward.cpp`
- Format A trit-budget sweep on Llama 1B: `tests/test_format_a_vs_b_llama.cpp`
- BitNet quality on Llama 1B (post-training): `tests/test_llama_bitnet_quality.cpp`
- Best CUDA packed kernel: `cuda/ter_cuda_packed_v6.cu`
- End-to-end CUDA forward (Llama, packed weights, 425 t/s): `cuda/ter_cuda_forward_packed.cu`
- End-to-end CUDA forward (BitNet 2B-4T real weights, 214 t/s, 8/8 bit-exact): `cuda/ter_cuda_forward_bitnet.cu`
- BitNet i2_s GGUF → packed converter: `tools/convert_bitnet_to_packed.py`
- Hybrid per-shape kernel dispatcher (M=1, 16, 64): `cuda/ter_cuda_hybrid_dispatch.cu`
- INT4 TC via `wmma::s4` fragments: `cuda/ter_cuda_int4_tc.cu`

Build (Mac AVX2):

```
cmake -S . -B build && cmake --build build
ctest --test-dir build --output-on-failure
```

Build CUDA bench (NVIDIA GPU + CUDA toolkit, ≥ Ampere SM 8.6):

```
nvcc -O3 -arch=sm_86 -std=c++17 cuda/ter_cuda_packed_v6.cu \
     -lcublas -o ter_cuda_packed_v6
./ter_cuda_packed_v6 100
```

Acknowledgments

The simulator borrows GGUF loading and tokenizer infrastructure from the author's prior `ntransformer` C++/CUDA Llama inference engine [4]. BitNet b1.58 architecture follows [8] and the `microsoft/BitNet` implementation [10]. `llama.cpp` [5] provided the production fp16 baseline measurements on the same RTX 3090. The CUDA `__dp4a` intrinsic is the core acceleration primitive enabling the $1.90\times$ win [13, 11].

References

[1] Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. GQA: Training generalized multi-query transformer models from multi-head checkpoints. *EMNLP*, 2023.

[2] N. P. Brusentsov. Setun: The first balanced ternary computer, 1959. Moscow State University, 1959. Historical predecessor of balanced-ternary computing architectures.

[3] Samuel Cortes. Brandon-tiny 10m: A 3-phase training pipeline for ultra-small instruction-following language models. 2026.

[4] Samuel Cortes. ntransformer: A c++/cuda inference engine for Llama 3.2 1B. 2026.

[5] Georgi Gerganov. llama.cpp: Inference of llama model in pure c/c++. https://github.com/ggerganov/llama.cpp, 2023. Accessed 2026-05-10.

[6] Georgi Gerganov. GGML quantization formats and GGUF file structure. 2024.

[7] John P. Hayes et al. Energy-efficient CMOS implementation of balanced ternary adders and multipliers. *IEEE Transactions on Circuits and Systems I*, 2018. Representative literature reference; multiple sources report 25–40% energy savings per equivalent-precision MAC for balanced ternary CMOS designs.

[8] Shuming Ma, Hongyu Wang, Lingxiao Ma, Lei Wang, Wenhui Wang, Shaohan Huang, Li Dong, Ruiping Wang, Jilong Xue, and Furu Wei. The era of 1-bit llms: All large language models are in 1.58 bits. *arXiv preprint arXiv:2402.17764*, 2024.

[9] Meta AI. Llama 3.2: Revolutionizing edge AI and vision with open, customizable models. https://ai.meta.com/blog/llama-3-2-connect-2024-vision-edge-mobile-devices/, 2024.

[10] Microsoft. BitNet: Official inference framework for 1-bit llms. https://github.com/microsoft/BitNet, 2024. Accessed 2026-05-10.

[11] NVIDIA. NVIDIA Ampere Architecture Whitepaper. https://images.nvidia.com/aem-dam/en-zz/Solutions/data-center/nvidia-ampere-architecture-whitepaper.pdf, 2020.

[12] NVIDIA. cuBLAS: NVIDIA GPU-accelerated basic linear algebra subroutines. https://docs.nvidia.com/cuda/cublas/, 2024.

[13] NVIDIA. CUDA C++ programming guide: Simd video instructions. https://docs.nvidia.com/cuda/cuda-c-programming-guide/, 2024. Accessed 2026-05-10.

[14] Noam Shazeer. GLU variants improve transformer. *arXiv preprint arXiv:2002.05202*, 2020.

[15] Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. RoFormer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568, 2024.

[16] Chetan Vudadha, A. Surya, Sruthi Agrawal, and M. B. Srinivas. A survey on ternary logic gate designs in CMOS technology. *IEEE Access*, 4:7530–7550, 2016.

[17] Hongyu Wang, Shuming Ma, Li Dong, Shaohan Huang, Huaijie Wang, Lingxiao Ma, Fan Yang, Ruiping Wang, Yi Wu, and Furu Wei. Bitnet: Scaling 1-bit transformers for large language models. *arXiv preprint arXiv:2310.11453*, 2023.

[18] Biao Zhang and Rico Sennrich. Root mean square layer normalization. *NeurIPS*, 2019.